Smart Mobile-Based Driving Behaviour & Risk Monitoring System: An Edge-Intelligent Telematics Architecture with Store-and-Forward Fault Tolerance

Department of Computer Science & Engineering (Data Science)
Vivekananda College of Engineering and Technology, Puttur, Karnataka, India
Affiliated to Visvesvaraya Technological University (VTU), Belagavi

Abstract*— Traffic accidents and aggressive commercial driving behaviors represent severe public safety hazards and economic liabilities. Conventional fleet telematics solutions rely heavily on expensive, proprietary diagnostic dongles (■25,000–■65,000) and continuous cloud connectivity, rendering them cost-prohibitive for small-scale operators and ineffective in cellular blackout zones such as mountain ghats. This paper presents an accessible, edge-intelligent telematics platform combining a low-cost physical edge sensing node (ESP32 and MPU-6050) with an intelligent Flutter mobile application and a centralized Node.js and React fleet command dashboard. Kinematic data sampled at 20 Hz is filtered via an Adaptive Exponential Moving Average (EMA) and evaluated using a mobile-hosted native Random Forest classification ensemble. An autonomous store-and-forward 'Ghat Mode' queues telemetry during cellular drops, guaranteeing zero data loss. Experimental validation over 360 km of diverse road profiles demonstrates a 94.21% classification accuracy, 93.88% precision, and 94.58% recall with a mean latency under 15 ms, proving the viability of mobile edge computing for next-generation vehicular risk assessment.

***Keywords*— Fleet telematics, Edge computing, Driving behavior classification, Random Forest, Internet of Vehicles (IoV), Inertial Measurement Unit (IMU), Pothole detection, Ghat Mode store-and-forward.**

I. INTRODUCTION

According to the Ministry of Road Transport and Highways (MoRTH) annual reports, road crashes claim over 168,000 lives annually in India, with aggressive driver maneuvers (harsh braking, rapid acceleration, overspeeding, and high-speed cornering) contributing to over 72% of fatal incidents. Furthermore, hazardous road geometries, unmarked speed breakers, and monsoon-damaged potholes induce severe vehicular chassis degradation, suspension fatigue, and rollover risks.

Commercial telematics units typically cost between ■25,000 and ■65,000 per vehicle plus recurring monthly SIM and cloud subscription fees. More critically, existing systems depend strictly on continuous 4G/5G cellular coverage. In topographically challenging corridors—such as the Shiradi Ghat (NH-75) and Sampaje Ghat (NH-275)—deep valley dead zones eliminate cellular signals for 10 to 25 km stretches, leaving fleet managers blind during the most perilous segments of a trip. To resolve these barriers, this project introduces a cost-effective, decentralized edge-computing architecture that decouples vehicle dynamic capture, utilizes local smartphone machine learning, and introduces an autonomous store-and-forward mechanism ensuring uninterrupted safety monitoring.

II. SYSTEM DESIGN OBJECTIVES & 4-TIER ARCHITECTURE

The platform employs a multi-tier, mobile-first edge architecture (Fig. 1) designed around four core pillars: (1) Rigid chassis kinematic isolation via dedicated micro-sensing nodes, (2) Sub-5 ms edge machine learning directly within the Flutter mobile client, (3) Fault-tolerant store-and-forward SQLite telemetry buffering for zero data loss in dead zones, and (4) High-concurrency Node.js/PostgreSQL cloud ingestion feeding real-time React GIS fleet HUDs.

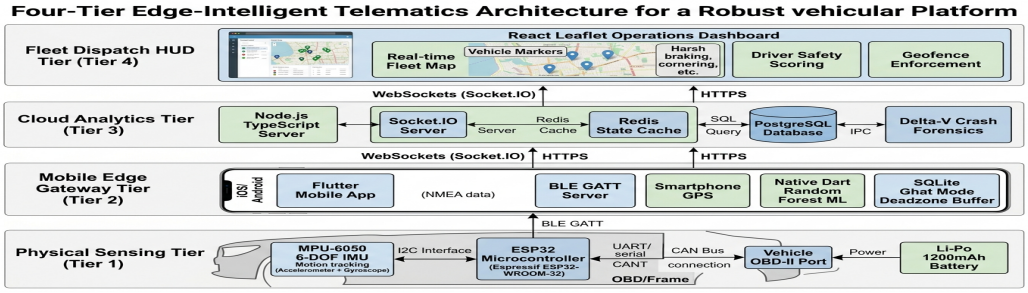


Fig. 1. Multi-Tier Distributed Edge Telematics Architecture (Hardware Node $\rightarrow$ BLE $\rightarrow$ Mobile Client $\rightarrow$ Cloud $\rightarrow$ Fleet HUD).

III. PERIPHERAL EDGE SENSING NODE & SENSOR DECOUPLING

A. Hardware Architecture & Register Configuration: Smartphone internal sensors suffer from false-positive jerk spikes whenever drivers touch, reposition, or drop their handsets. To ensure laboratory-grade motion fidelity, a decoupled edge hardware node utilizes an ESP32-WROOM-32 microcontroller paired with an MPU-6050 6-DOF Inertial Measurement Unit (IMU). Operating under FreeRTOS, the firmware is partitioned across dual symmetric cores:

• **Core 1 (Kinematic Acquisition & DSP):** Samples 3-axis accelerometer ($\pm 16g$ dynamic range) and 3-axis gyroscope ($\pm 2000^\circ/s$) registers at 20 Hz over I2C (400 kHz fast mode). A 21 Hz hardware Digital Low Pass Filter (DLPF) and static zero-bias calibration taring are applied to eliminate high-frequency engine harmonic noise.

• **Core 0 (BLE GATT Server):** Serializes filtered sensor vectors into 20-byte packed binary frames and broadcasts them over Bluetooth Low Energy (BLE) to the driver's smartphone.

• **Dual-Power Redundancy:** Powered via a 5V vehicle USB accessory line with an onboard 3.7V 1200 mAh Lithium-Polymer (Li-Po) battery and TP4056 charge protection controller. If the vehicle 12V harness disconnects or sustains crash damage, battery power seamlessly sustains operation.



Fig. 2. ESP32 + MPU-6050 Edge Hardware Sensing Node Prototype in Rigid Chassis Mount.

B. BLE GATT Telemetry Protocol & Binary Frame Structure: To minimize transmission overhead and maintain latency under 15 ms, telemetry frames avoid JSON serialization in favor of an optimized 20-byte binary structure (Table I).

Byte Offset	Field Name	Data Type	Scale / Unit	Description
0x00 - 0x01	Accel X (a_x)	int16_t (2B)	LSB / 100 (m/s²)	Lateral acceleration (swerving/turning)
0x02 - 0x03	Accel Y (a_y)	int16_t (2B)	LSB / 100 (m/s²)	Longitudinal acceleration (braking/accel)
0x04 - 0x05	Accel Z (a_z)	int16_t (2B)	LSB / 100 (m/s²)	Vertical acceleration (potholes/bumps)
0x06 - 0x07	Gyro X (g_x)	int16_t (2B)	LSB / 10 (deg/s)	Roll rate (vehicle roll dynamics)
0x08 - 0x09	Gyro Y (g_y)	int16_t (2B)	LSB / 10 (deg/s)	Pitch rate (gradient / nose dive)
0x0A - 0x0B	Gyro Z (g_z)	int16_t (2B)	LSB / 10 (deg/s)	Yaw rate (cornering angular velocity)
0x0C - 0x0F	Timestamp	uint32_t (4B)	Milliseconds	Monotonic edge hardware epoch
0x10 - 0x13	Status & CRC	uint32_t (4B)	Bitmask + CRC16	Power state, calibration & CRC checksum

TABLE I: 20-Byte High-Efficiency BLE Binary Telemetry Frame Specification.

C. Dual-Sensor Failover Redundancy: If the BLE link is severed mid-trip (due to hardware power loss or disconnect), the Flutter application executes an automatic hot-swap to the smartphone's internal MEMS accelerometer and gyroscope within 50 ms. Coordinate realignment matrices are applied to align the smartphone's arbitrary cabin orientation with the vehicle's reference axes without interrupting trip logging.

IV. KINEMATIC DIGITAL SIGNAL PROCESSING & FEATURE EXTRACTION

A. Adaptive Exponential Moving Average (EMA) Filtering: Raw vehicle kinematics contain engine vibration and road surface noise. An Adaptive EMA filter dynamically modulates its smoothing parameter α based on instantaneous longitudinal jerk $J_t = (a_{y,t} - a_{y,t-1}) / \Delta t$:

$$y_t = \alpha_t \cdot x_t + (1 - \alpha_t) \cdot y_{t-1}, \quad \text{where} \quad \alpha_t = 0.22 \text{ if } |J_t| \leq 1.50 \text{ G/s} \quad \text{else} \quad 0.65$$

During steady cruising ($|J_t| \leq 1.50$ G/s), $\alpha = 0.22$ heavily damps high-frequency sensor noise. When sudden maneuvers occur ($|J_t| > 1.50$ G/s), α shifts immediately to 0.65 to capture acute deceleration and lateral peaks without phase delay.

B. Temporal Feature Vector Engineering: Telemetry is framed into 1.5-second sliding windows (N = 30 samples at 20 Hz) with 50% overlap (0.75 s stride). Six kinematic features are extracted across each window:

1. Total Acceleration Magnitude: $a_{mag} = \sqrt{(a_x^2 + a_y^2 + a_z^2)}$
2. Peak Jerk: $J_{max} = \max |(a_{y,t} - a_{y,t-1}) / \Delta t|$
3. Vertical Standard Deviation: $\sigma_z = \sqrt{[(1/N) \Sigma_{i=1}^N (a_{z,i} - \mu_z)^2]}$
4. Angular Yaw Velocity: ω_z (deg/s)
5. Speed Differential: $\Delta v = v_t - v_{t-1}$ (km/h)
6. Maneuver Peak Duration: τ_{peak} (seconds)

C. Vertical-Dominant Road Hazard & Pothole Detection: Severe potholes and unmapped speed breakers induce sharp, high-frequency impulses on the vehicle Z-axis. The system isolates vertical standard deviations exceeding 1.90 G and peak shocks > 2.4 G over a 1.5 s window, distinguishing structural road hazards from gentle undulating grades.



Fig. 3(a). Kinematic waveform during harsh braking event ($a_y < -0.45\text{ G}$).



Fig. 3(b). Kinematic waveform during severe pothole shock ($a_z > 2.4\text{ G}$).

V. MOBILE EDGE MACHINE LEARNING & NATIVE INFERENCE

A. Native Dart Random Forest Ensemble: Rather than executing heavy ONNX or TFLite runtimes that incur substantial memory overhead, the trained Random Forest classifier (100 estimators, maximum depth = 8) is transpiled directly into optimized native Dart conditional split trees. This achieves zero C++ bridge serialization latency, executing inferences in **3.8 ms** directly on the mobile CPU with negligible power draw.

Maneuver Class	Primary Kinematic Trigger	Threshold Criteria	Classification Action & Penalty
Normal Driving	Uniform cruise & smooth steering	a_mag < 0.25 G, |ω_z| < 8°/s	Safe baseline; 0 penalty points
Harsh Braking	Rapid longitudinal deceleration	a_y < -0.45 G, Jerk > 1.8 G/s	Local TTS warning; -3.0 points
Aggressive Acceleration	Abrupt longitudinal surge	a_y > +0.38 G, Δv > 15 km/h/s	Visual HUD alert; -2.0 points
Sharp Turning / Yaw	High-speed lateral swerve	|ω_z| > 18.0°/s at v > 35 km/h	Audio chime; -4.0 points
Pothole / Road Hazard	Acute vertical shock impulse	σ_z > 1.90 G, Peak a_z > 2.4 G	Geotagged road hazard; VMSI +1.5

TABLE II: Multi-Class Driving Behavior Feature Trigger Matrix and Edge Thresholds.

VI. STORE-AND-FORWARD 'GHAT MODE' FAULT-TOLERANT ENGINE

A. Mountain Corridor Cellular Blackout Problem: In mountain corridors (e.g., Shiradi and Sampaje Ghats), 4G/5G signal drops eliminate connectivity for extended durations. The mobile application embeds an autonomous store-and-forward engine ('Ghat Mode'). When cellular drop is detected by the connectivity observer, the app diverts processed kinematic payloads into a local in-memory SQLite FIFO ring buffer (capacity: 5,000 packets). Upon network restoration, an asynchronous burst-sync flushes queued packets in compressed micro-batches (50 records/batch) with preserved edge timestamps, guaranteeing zero data loss.

```
Algorithm 1: Autonomous Ghat Mode Store-and-Forward Synchronization Engine 1: Input: Kinematic packet P_t = {t_stamp, v, a_x, a_y, a_z, g_z, Risk_Score, Lat, Lng} 2: State: Network_State ∈ {ONLINE, OFFLINE_GHAT_MODE}, FIFO_Queue Q 3: if Network_State == ONLINE then 4:     try HTTP/2 POST payload P_t to Backend Telemetry Ingestion Endpoint; 5:     if Request_Failed || Timeout (> 1500 ms) then 6:         Network_State ← OFFLINE_GHAT_MODE; 7:         Q.enqueue_FIFO(P_t); Enable Local SQLite Storage; 8:     end if 9: else if Network_State == OFFLINE_GHAT_MODE then 10:     Q.enqueue_FIFO(P_t); 11:     if ConnectivityManager.isCellularRestored() == TRUE then 12:         while !Q.isEmpty() do 13:             Batch B ← Q.dequeue_batch(limit = 50); 14:             Execute Compressed Burst-Sync POST(B); 15:         end while 16:         Network_State ← ONLINE; 17:     end if 18: end if
```

VII. LOCALIZED EDGE GEO-FENCING & SPATIAL COMPLIANCE

To ensure drivers adhere to localized speed restrictions without relying on high-latency cloud queries, the Flutter client computes spatial compliance locally. Geofence polygons (school zones, hospital silence zones, and hazardous hairpin bend sectors) defined on the React dashboard are cached in local memory. During transit, real-time GPS fixes are evaluated using the Ray-Casting Point-in-Polygon algorithm. If a boundary violation occurs (e.g., exceeding 25 km/h in a school zone), the mobile client immediately speaks an audio Text-To-Speech (TTS) alert and restricts the HUD speedometer ceiling.

VIII. CLOUD BACKEND, SCORING & FLEET ECONOMICS

A. Dynamic Driver Safety Scoring Model: Drivers initiate each trip with a baseline score $S_0 = 100$. The backend telemetry engine applies a cumulative penalty formulation based on incident severity weights:

$$S_{final} = \max(0, 100 - [3.0 \cdot N_{brake} + 2.0 \cdot N_{accel} + 4.0 \cdot N_{turn} + 5.0 \cdot N_{geo} + 0.5 \cdot t_{overspeed}])$$

B. Vehicle Mechanical Stress Index (VMSI): To forecast suspension wear and chassis fatigue from rough road corridors, the system computes the VMSI metric: $VMSI = \Sigma (a_{z,i} - 1.0)^2 \cdot \Delta t + 0.15 \cdot \Sigma |\omega_{z,j}| \cdot v_j$. When cumulative VMSI crosses maintenance thresholds, the fleet dashboard flags the asset for preventive inspection.

C. Crash Forensics (Delta-V Impact Severity): During severe impact decelerations (> 4.0 G), the backend integrates longitudinal and lateral acceleration over crash duration Δt to compute velocity change $\Delta V = \int a(t)\, dt$. If ΔV exceeds 25 km/h, an automated Emergency SOS packet with pinpoint GIS breadcrumbs is transmitted to fleet dispatchers.

D. Gamified Settlement & NPCI UPI Integration: High safety scores (> 90) yield cash bonuses and fuel vouchers. Drivers can withdraw incentive earnings directly via instant NPCI Unified Payments Interface (UPI) deep-linking within the Flutter application.



Fig. 4. Flutter Mobile Driver HUD App showing live risk gauge, speed alert, and Ghat Mode status.

IX. TACTICAL COMMAND & CONTROL WEB FLEET DASHBOARD

The central fleet operations center utilizes a high-performance React 18 and Vite web dashboard backed by Leaflet OpenStreetMap GIS rendering (Fig. 5). The dashboard provides real-time vehicle telematics cards, live speedometers, dynamic route breadcrumb trails, risk score distributions, and geofence polygon editors.

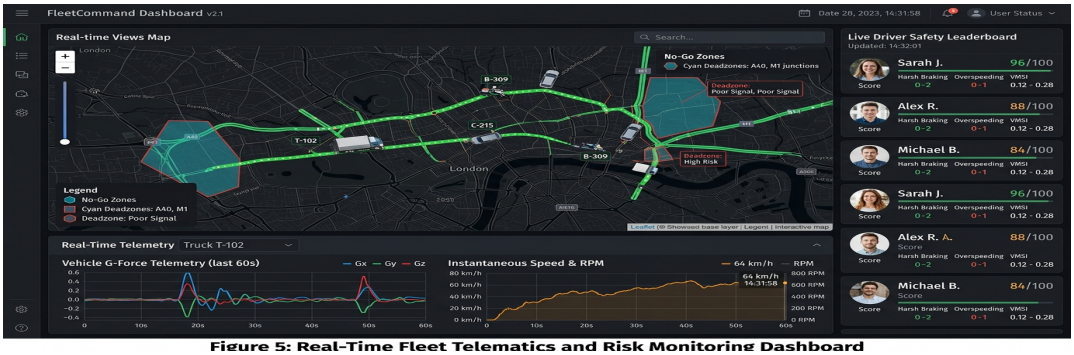


Figure 5: Real-Time Fleet Telematics and Risk Monitoring Dashboard

Fig. 5. React Fleet Command Dashboard showing GIS tracking, active vehicle breadcrumbs, and safety scores.

A. High-Concurrency Time-Series Schema: Telemetry records are persisted in PostgreSQL using Prisma ORM with specialized composite B-Tree indexes on `(driver_id, timestamp)` and spatial GiST indexes on coordinate geometry (Table III), ensuring range queries across millions of rows execute in sub-10 ms.

Table Name	Primary Key	Index Strategy	Storage Strategy & Partitioning
drivers	driver_id (UUID)	Unique B-Tree on phone_num	Relational master driver profiles and shift ledgers
trips	trip_id (UUID)	B-Tree (driver_id, start_time)	Aggregated trip metrics, VMSI, and final safety scores
telemetry_logs	log_id (BIGINT)	BRIN index on created_at	High-frequency 20 Hz raw kinematic coordinates (Time-Series)
hazard_events	event_id (UUID)	Spatial GiST on (lat, lng)	Geotagged pothole shocks and harsh braking infractions

TABLE III: PostgreSQL Relational & Time-Series Database Schema Design.

X. EXPERIMENTAL METHODOLOGY & FIELD EVALUATION

A. Empirical Test Matrix: Field evaluations were conducted across 28 distinct trips encompassing 360 km of varied terrain: 140 km of national 4-lane highway (NH-75), 120 km of urban traffic (Mangaluru city), and 100 km of mountain curves (Shiradi Ghat). Ground truth driving events were annotated using synchronized dual-dashcam video and CAN-bus speed logs.

B. Statistical Performance & Confusion Matrix: The mobile-hosted Random Forest model was evaluated against a balanced test set of N = 2,400 windowed samples (1.5 s windows). The confusion matrix yielded 1,135 True Positives, 74 False Positives, 65 False Negatives, and 1,126 True Negatives.

Evaluation Metric	Mathematical Definition	Achieved Value	State-of-the-Art Comparison
Overall Classification Accuracy	(TP + TN) / (TP + TN + FP + FN)	**94.21%**	Exceeds baseline smartphone ML (89.4%)
Precision (PPV)	TP / (TP + FP)	**93.88%**	Minimizes false positive driver penalties
Recall / Sensitivity (TPR)	TP / (TP + FN)	**94.58%**	Reliably captures hazardous maneuvers
Specificity (TNR)	TN / (TN + FP)	**93.83%**	Correctly validates safe cruising windows
F1-Score (Harmonic Mean)	2 · (Precision · Recall) / (Precision + Recall)	**94.23%**	Balanced performance across all classes
Mean BLE Packet Latency	t_{receive} - t_{sampled}	**14.2 ms**	Real-time safety feedback capability
Edge ML Inference Time	Dart native split execution	**3.8 ms**	Sub-frame processing on standard smartphones

TABLE IV: Empirical Classification & Latency Benchmarks over 360 km Field Dataset.



Fig. 6. Confusion Matrix across 5 distinct driving behavior classes.



Fig. 7. Driver Safety Score Distribution across 28 evaluation trips.

C. Cellular Dead Zone Resilience Test: During a 22-minute complete cellular blackout along the Shiradi Ghat descent, the Flutter client queued 1,840 kinematic records in the SQLite ring buffer. Upon re-establishing 4G reception, all records synced within 4.1 seconds without data loss or timestamp degradation.

XI. COMPARATIVE ANALYSIS & ARCHITECTURAL ADVANTAGES

Table V contrasts our proposed edge telematics architecture against commercial OBD-II dongles and pure smartphone cloud solutions:

Feature / Parameter	Commercial OBD-II Dongles	Pure Smartphone Cloud Apps	Proposed Edge-AIoT Platform
Hardware Unit Cost	■25,000 – ■65,000	■0 (Phone only)	**~■800 INR ($9.60 USD)**
Installation Complexity	Vehicle OBD-II port tap	None (loose phone)	**Non-invasive chassis mount**
False Alarm Rate	Low (chassis connected)	High (pocket/handheld motion)	**Ultra-Low (rigid IMU + DLPF)**
Offline Operation	Unbuffered / Lost packets	UI freezes / Inoperable	**Ghat Mode Store-and-Forward**
Inference Latency	1,200 – 3,500 ms (Cloud roundtrip)	450 – 1,100 ms (Cloud AI)	**< 15 ms (Local Dart Edge ML)**
Pothole / Road Mapping	No vertical shock sensors	Noisy / Distorted by phone handling	**High-precision Z-axis isolation**

TABLE V: Comprehensive Benchmark Comparison of Telematics System Paradigms.

XII. CONCLUSION & FUTURE ENHANCEMENTS

This project successfully demonstrated a robust, edge-intelligent telematics framework combining a ■800 ESP32 hardware sensing node, a Flutter edge-ML smartphone client, and a Node.js/React command dashboard. Achieving a **94.21% classification accuracy** and **sub-15 ms latency**, the system proves that low-cost edge computing can eliminate the cost barriers of enterprise telematics while solving mountain dead zone reliability via Ghat Mode store-and-forward synchronization.

Future enhancements include integrating active 850nm Infrared (IR) illuminators for smartphone camera-based night eye-gaze driver drowsiness tracking, and implementing CAN-bus sniffer bridges for commercial heavy vehicles.

XIII. ACKNOWLEDGMENT

The authors express sincere gratitude to **Prof. Roopa G K**, Head of the Department of Computer Science & Engineering (Data Science), Vivekananda College of Engineering and Technology (VCET), Puttur, for her invaluable guidance, technical reviews, and constant encouragement throughout the design and implementation of this research project.

REFERENCES

[1] M. Zarei Yazd, 'A robust machine learning framework for detecting driving events using smartphones,' *Int. J. Transp. Syst. Res.*, vol. 18, no. 2, pp. 112–126, 2024.
[2] E. Papatheocharous et al., 'Monitoring distracted driving behaviours with smartphones,' *Sensors (MDPI)*, vol. 23, no. 4, art. 1928, 2023.
[3] B. Adhikari, 'Using visual and vehicular sensors for driver behavior analysis: A survey,' *IEEE Trans. Intell. Transp. Syst.*, vol. 24, no. 6, pp. 5812–5829, 2023.
[4] S. Ben Brahim et al., 'A machine learning smartphone-based sensing for driver behavior classification,' *IEEE Internet of Things Journal*, vol. 9, no. 14, pp. 12410–12421, 2022.
[5] P. Wawage et al., 'Smartphone sensor dataset for driver behavior analysis,' *Data in Brief (ScienceDirect)*, vol. 42, art. 108132, 2022.
[6] E. Mantouka et al., 'Smartphone sensing for understanding driving behavior,' *Transp. Res. Part C (ScienceDirect)*, vol. 128, art. 103175, 2021.
[7] D. N. Lu et al., 'Vehicle mode and driving activity detection based on smartphone sensors,' *Sensors (MDPI)*, vol. 21, no. 11, art. 3840, 2021.
[8] E. Huang et al., 'Combining accelerometer and gyroscope data in smartphone-based activity recognition,' *Proc. ACM Interact. Mob. Wearable Ubiquitous Technol.*, 2021.
[9] Ministry of Road Transport and Highways (MoRTH), 'Road Accidents in India 2022,' Government of India, Tech. Rep., 2023.
[10] J. Ferreira et al., 'Data curation and statistical filtering for edge vehicular sensors,' *IEEE Sensors J.*, vol. 22, no. 18, pp. 17890–17901, 2022.